

Scientific Python - Pandas

A.Y 2020/2021

What is Pandas

- Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures.
- The name Pandas is derived from the word Panel Data – an Econometrics term for data sets that include observations over multiple time periods for the same individuals.

Series and a DataFrame

- In pandas, we have two main data structures that we can explore. The first is a DataFrame and the second is a Series. So what's the different between the two?
- A DataFrame is a two-dimensional array of values with both a row and a column index.
- A Series is a one-dimensional array of values with an index.

Series and a DataFrame (contd.)

Series

	Value
0	NJ
1	CA
2	TX
3	MD
4	OH
5	IL

DataFrame

	Column Index		
	State	City	Shape
0	NJ	Towaco	Square
1	CA	San Francisco	Oval
2	TX	Austin	Triangle
3	MD	Baltimore	Square
4	OH	Columbus	Hexagon
5	IL	Chicaco	Circle

- Where a **DataFrame** is the entire dataset, including all rows and columns — a **Series** is essentially a single column within that **DataFrame**. Creating these two data structures is a fairly straightforward process in pandas.

Creating Series and DataFrames

```
import pandas as pd

df = pd.DataFrame(data = [
    ['NJ', 'Towaco', 'Square'],
    ['CA', 'San Francisco', 'Oval'],
    ['TX', 'Austin', 'Triangle'],
    ['MD', 'Baltimore', 'Square'],
    ['OH', 'Columbus', 'Hexagon'],
    ['IL', 'Chicago', 'Circle']
],
    columns = ['State', 'City', 'Shape'])

s = pd.Series(data=['NJ', 'CA', 'TX', 'MD', 'OH', 'IL'])
```

Column Index			
	State	City	Shape
0	NJ	Towaco	Square
1	CA	San Francisco	Oval
2	TX	Austin	Triangle
3	MD	Baltimore	Square
4	OH	Columbus	Hexagon
5	IL	Chicaco	Circle

Value	
0	NJ
1	CA
2	TX
3	MD
4	OH
5	IL

Useful methods and properties

- **.head(n_rows):** returns a new DataFrame composed of the first n_rows rows.
The parameter n_rows is optional and it is set to 5 by default
- **.tail(n_rows):** returns a new DataFrame composed of the last n_rows rows.
The parameter n_rows is optional and it is set to 5 by default
- **.shape:** returns the shape of the DataFrame that provides the number of elements for both the dimensions of the DataFrame
- **.index:** returns the labels of the DataFrame indexes
- **.to_numpy():** converts the DataFrame to a NumPy array
- **.describe():** shows a quick statistic summary of your data.

Useful methods and properties (contd.)

```
import pandas as pd

df = pd.DataFrame(data = [
    ['NJ', 'Towaco', 'Square'],
    ['CA', 'San Francisco', 'Oval'],
    ['TX', 'Austin', 'Triangle'],
    ['MD', 'Baltimore', 'Square'],
    ['OH', 'Columbus', 'Hexagon'],
    ['IL', 'Chicago', 'Circle']
],
    columns = ['State', 'City', 'Shape'])

print(df.head(2))
print(df.tail(3))
print(df.shape)
print(df.to_numpy())
print(df.describe())
```

Data selection

```
import pandas as pd

df = pd.DataFrame(data = [
    ['NJ', 'Towaco', 'Square'],
    ['CA', 'San Francisco', 'Oval'],
    ['TX', 'Austin', 'Triangle'],
    ['MD', 'Baltimore', 'Square'],
    ['OH', 'Columbus', 'Hexagon'],
    ['IL', 'Chicago', 'Circle']
],
    columns = ['State', 'City', 'Shape'])

series = df['State'] # by label
sliced_df = df[1:4] # getting a slice
multiaxis_slice = df.loc[1:3, ['State', 'City']] # slice by label
multiaxis_slice_iloc = df.iloc[1:3, 0:2] # slice by position
```


Arithmetical and statistical methods

```
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randn(6, 4), columns=list('ABCD'))

df_mean = df.mean() # Mean column per column
df_max = df.max() # Max value in each column
df_min = df.min() # Min value in each column
df_sum = df.sum() # Sum of the values in each column
df_count = df.count() # Count non-NA cells for each column or row.
df_diff = df.diff() # First discrete difference of element

# standard correlation coefficient.
# Other possible methods are: 'kendall' and 'spearman'
df_corr = df.corr(method='pearson')
```

Read DataFrame from CSV

- A DataFrame object can be read from a CSV with the method `auto = pd.read_csv(file)`

```
import pandas as pd

df = pd.read_csv("Auto.csv", delimiter=",")

print(df)
```

Sorting values

pandas.DataFrame.sort_values

```
DataFrame.sort_values(self, by, axis=0, ascending=True, inplace=False, kind='quicksort', na_position='last')
```

Sort by the values along either axis.

[\[source\]](#)

Parameters:

by : str or list of str

Name or list of names to sort by.

- if *axis* is 0 or 'index' then *by* may contain index levels and/or column labels
- if *axis* is 1 or 'columns' then *by* may contain column levels and/or index labels

Changed in version 0.23.0: Allow specifying index or column level names.

axis : {0 or 'index', 1 or 'columns'}, default 0

Axis to be sorted.

ascending : bool or list of bool, default True

Sort ascending vs. descending. Specify list for multiple sort orders. If this is a list of bools, must match the length of the by.

inplace : bool, default False

If True, perform operation in-place.

kind : {'quicksort', 'mergesort', 'heapsort'}, default 'quicksort'

Choice of sorting algorithm. See also `ndarray.sort` for more information. *mergesort* is the only stable algorithm. For DataFrames, this option is only applied when sorting on a single column or label.

na_position : {'first', 'last'}, default 'last'

Puts NaNs at the beginning if *first*; *last* puts NaNs at the end.

Sorting indexes

pandas.DataFrame.sort_index

```
DataFrame.sort_index(self, axis=0, level=None, ascending=True, inplace=False, kind='quicksort',  
na_position='last', sort_remaining=True, by=None)
```

[\[source\]](#)

Sort object by labels (along an axis).

Parameters:

axis : {0 or 'index', 1 or 'columns'}, default 0

The axis along which to sort. The value 0 identifies the rows, and 1 identifies the columns.

level : int or level name or list of ints or list of level names

If not None, sort on values in specified index level(s).

ascending : bool, default True

Sort ascending vs. descending.

inplace : bool, default False

If True, perform operation in-place.

kind : {'quicksort', 'mergesort', 'heapsort'}, default 'quicksort'

Choice of sorting algorithm. See also `ndarray.sort` for more information. *mergesort* is the only stable algorithm. For DataFrames, this option is only applied when sorting on a single column or label.

na_position : {'first', 'last'}, default 'last'

Puts NaNs at the beginning if *first*; *last* puts NaNs at the end. Not implemented for MultiIndex.

sort_remaining : bool, default True

If True and sorting by level and index is multilevel, sort by other levels too (in order) after sorting by specified level.

Group By

- A group by operation involves some combination of splitting the object, applying a function, and combining the results. This can be used to group large amounts of data and compute operations on these groups.

```
import pandas as pd

df = pd.DataFrame({'Animal': ['Falcon', 'Falcon', 'Parrot', 'Parrot'],
                  'Max Speed': [380., 370., 24., 26.]})

print(df.groupby(['Animal']).count())
```

Exercise

1. Create a class that provides the `trio.sample.vcf` dataset as a `DataFrame` and allows to count the number of occurring bases for each available chromosome.

Exercise on the Iris dataset

- Given the iris dataset: <https://gist.githubusercontent.com/curran/a08a1080b88344b0c8a7/raw/639388c2cbc2120a14dcf466e85730eb8be498bb/iris.csv>
- Write an Object-Oriented program that has a responsible for reading the dataset, then:
 1. Provides the number of rows and columns it contains
 2. Computes the average petal length
 3. Computes the average of all numerical columns
 4. Extracts the petal length outliers (i.e. those rows whose petal length is 50% longer than the average petal length)
 5. Computes the standard deviation of all columns, for each iris species
 6. Extracts the petal length outliers (as above) for each iris species
 7. Extracts the group-wise petal length outliers, i.e. find the outliers (as above) for each iris species using `groupby()`, `aggregate()`, and `merge()`.